

CORE CSE SERIES — MCQ EDITION

Data Structures & Algorithms MCQ Handbook

Basic to Advanced Multiple-Choice Questions for Company OTs, PYQ-style Practice & Core Placement Preparation — with Answers & Explanations

Complexity

Arrays & Lists

Stacks/Queues/Hashing

Trees & Graphs

Sorting, Searching & DP

Table of Contents — Data Structures & Algorithms

- 1. Complexity Analysis** (5 questions)
- 2. Arrays, Strings & Linked Lists** (6 questions)
- 3. Stacks, Queues & Hashing** (6 questions)
- 4. Trees & Graphs** (9 questions)
- 5. Sorting, Searching & Dynamic Programming** (8 questions)

Topic-wise MCQs with Explanations

1. Complexity Analysis Big-O, Big-Omega, Big-Theta, amortized analysis

Q1 What does Big-O notation describe?

BASIC

- A. The upper bound / worst-case growth rate of an algorithm's time or space
- B. The exact runtime of an algorithm in seconds
- C. Only the best-case performance
- D. The amount of RAM a computer has

Correct Answer: A. The upper bound / worst-case growth rate of an algorithm's time or space

Explanation: Big-O describes an asymptotic upper bound on growth as input size increases, ignoring constant factors and lower-order terms.

Q2 What is the time complexity of binary search on a sorted array of size n ?

BASIC

- A. $O(\log n)$
- B. $O(n)$
- C. $O(n \log n)$
- D. $O(1)$

Correct Answer: A. $O(\log n)$

Explanation: Binary search halves the search space each step, giving logarithmic time complexity, $O(\log n)$.

Q3 What is the worst-case time complexity of quicksort?

INTERMEDIATE

- A. $O(n^2)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(\log n)$

Correct Answer: A. $O(n^2)$

Explanation: Quicksort's worst case (e.g., already sorted input with a poor pivot choice) is $O(n^2)$, though its average case is $O(n \log n)$.

Q4 Which of the following correctly orders these complexities from fastest to slowest growth?

BASIC

- A. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- B. $O(2^n) < O(n^2) < O(n \log n) < O(n) < O(\log n) < O(1)$
- C. $O(n) < O(\log n) < O(1) < O(n^2)$
- D. $O(n \log n) < O(\log n) < O(n) < O(1)$

Correct Answer: A. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

Explanation: This is the standard ordering of common complexity classes by growth rate, from constant to exponential.

Q5 What is amortized time complexity?

ADVANCED

- A. The average time per operation over a sequence of operations, even if some individual operations are costly
- B. The best-case time complexity only
- C. The time complexity ignoring space usage
- D. A synonym for worst-case complexity

Correct Answer: A. The average time per operation over a sequence of operations, even if some individual operations are costly

Explanation: Amortized analysis averages the cost across a sequence of operations (e.g., dynamic array doubling) so that occasional expensive operations don't dominate the per-operation cost.

2. Arrays, Strings & Linked Lists Two pointers, sliding window, list operations

Q6 What is the time complexity of inserting an element at the beginning of an array of size n ?

BASIC

- A. $O(n)$, because all existing elements must shift
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A. $O(n)$, because all existing elements must shift

Explanation: Inserting at the start of an array requires shifting all n existing elements one position to the right, giving $O(n)$ time.

Q7 What is the time complexity of inserting a node at the head of a singly linked list?

BASIC

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A. $O(1)$

Explanation: Since the head reference can be updated directly without traversal, inserting at the head of a linked list is $O(1)$.

Q8 The 'two-pointer' technique is most useful for problems involving:

INTERMEDIATE

- A. Sorted arrays or sequences where you need to find pairs/subarrays satisfying a condition
- B. Only unsorted hash maps
- C. Tree traversal exclusively
- D. Only recursive backtracking

Correct Answer: A. Sorted arrays or sequences where you need to find pairs/subarrays satisfying a condition

Explanation: Two pointers (often starting from both ends or moving at different speeds) efficiently solve problems like pair-sum in a sorted array or detecting cycles, often in $O(n)$.

Q9 What technique is typically used to solve 'maximum sum subarray of size k' efficiently?

INTERMEDIATE

- A. Sliding window
- B. Binary search
- C. Depth-first search
- D. Dynamic programming with $O(n^2)$ states

Correct Answer: A. Sliding window

Explanation: The sliding window technique maintains a running sum over a fixed-size window, adding the new element and removing the old one in $O(1)$ per step, giving $O(n)$ total.

Q10 In a doubly linked list compared to a singly linked list, the main advantage is:

BASIC

- A. It allows traversal in both directions (forward and backward)
- B. It uses less memory
- C. It has $O(1)$ random access
- D. It cannot have a cycle

Correct Answer: A. It allows traversal in both directions (forward and backward)

Explanation: A doubly linked list maintains both 'next' and 'prev' pointers per node, allowing bidirectional traversal at the cost of extra memory per node.

Q11 What is the time complexity to reverse a singly linked list of n nodes iteratively?

BASIC

- A. $O(n)$
- B. $O(1)$
- C. $O(n^2)$
- D. $O(\log n)$

Correct Answer: A. $O(n)$

Explanation: Reversing requires visiting each node once to redirect its pointer, giving linear time $O(n)$ with $O(1)$ extra space.

3. Stacks, Queues & Hashing Stack/queue applications, hash table design

Q12 Which data structure is most naturally used to check for balanced parentheses in an expression?

BASIC

- A. Stack
- B. Queue
- C. Hash Table
- D. Graph

Correct Answer: A. Stack

Explanation: A stack tracks opening brackets; when a closing bracket appears, it's matched against the top of the stack, making stacks the natural fit for this problem.

Q13 A circular queue solves which limitation of a simple array-based linear queue?

INTERMEDIATE

- A. It reuses freed space at the front instead of wasting it after dequeue operations
- B. It removes the need for a rear pointer
- C. It allows $O(1)$ search
- D. It eliminates the need for enqueue/dequeue operations

Correct Answer: A. It reuses freed space at the front instead of wasting it after dequeue operations

Explanation: In a linear queue, dequeuing from the front leaves unusable space; a circular queue wraps around to reuse that freed space, improving space utilization.

Q14 What is the average time complexity for search, insert, and delete in a well-designed hash table? **BASIC**

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

Correct Answer: A. $O(1)$

Explanation: With a good hash function and load factor, hash table operations average $O(1)$, though worst case (many collisions) can degrade to $O(n)$.

Q15 Which of the following is a common technique to resolve hash collisions? **INTERMEDIATE**

- A. Chaining (linked lists at each bucket) or open addressing (probing)
- B. Always resizing the array to infinite size
- C. Ignoring the collision
- D. Sorting the hash table

Correct Answer: A. Chaining (linked lists at each bucket) or open addressing (probing)

Explanation: Chaining stores multiple colliding entries in a list/tree per bucket; open addressing finds another empty slot within the table itself via probing.

Q16 What is a 'load factor' in a hash table? **INTERMEDIATE**

- A. The ratio of the number of stored elements to the total number of buckets
- B. The maximum size of the hash table
- C. The number of collisions that occurred
- D. The time taken to compute a hash

Correct Answer: A. The ratio of the number of stored elements to the total number of buckets

Explanation: Load factor = n/m (elements/buckets); as it approaches or exceeds 1, collision likelihood rises, often triggering a resize/rehash.

Q17 Which application commonly uses a queue (FIFO) rather than a stack? **BASIC**

- A. Breadth-First Search (BFS) traversal
- B. Depth-First Search (DFS) traversal
- C. Function call stack management
- D. Undo/redo functionality

Correct Answer: A. Breadth-First Search (BFS) traversal

Explanation: BFS explores nodes level by level, requiring FIFO order to process nodes in the order they were discovered — a classic queue use case.

4. Trees & Graphs BST, traversals, graph representations, shortest path

Q18 In a Binary Search Tree, an in-order traversal visits nodes in which order?

BASIC

- A. Ascending sorted order
- B. Descending sorted order
- C. Random order
- D. Level order

Correct Answer: A. Ascending sorted order

Explanation: In-order traversal (Left → Node → Right) of a BST always visits nodes in ascending sorted order due to the BST property.

Q19 What is the height of a balanced binary tree with n nodes?

BASIC

- A. $O(\log n)$
- B. $O(n)$
- C. $O(1)$
- D. $O(n^2)$

Correct Answer: A. $O(\log n)$

Explanation: A balanced binary tree keeps left and right subtree heights close, resulting in a height proportional to $\log n$ rather than n .

Q20 Which traversal is used to create a copy of a binary tree (visiting node before its children)?

INTERMEDIATE

- A. Pre-order
- B. In-order
- C. Post-order
- D. Level-order only

Correct Answer: A. Pre-order

Explanation: Pre-order (Node → Left → Right) visits the parent before its children, which is useful for cloning a tree or serializing its structure.

Q21 Which graph representation is more space-efficient for a sparse graph?

INTERMEDIATE

- A. Adjacency list**
- B. Adjacency matrix
- C. Both are always equal
- D. Incidence matrix always

Correct Answer: A. Adjacency list

Explanation: An adjacency list uses $O(V+E)$ space, ideal for sparse graphs; an adjacency matrix always uses $O(V^2)$ regardless of the actual number of edges.

Q22 Which algorithm finds the shortest path from a single source in a graph with non-negative edge weights?

INTERMEDIATE

- A. Dijkstra's Algorithm**
- B. Bellman-Ford only
- C. Kruskal's Algorithm
- D. DFS

Correct Answer: A. Dijkstra's Algorithm

Explanation: Dijkstra's algorithm efficiently computes shortest paths from a source when all edge weights are non-negative, typically using a min-heap/priority queue.

Q23 Which algorithm can handle graphs with negative edge weights (but no negative cycles) for shortest path?

ADVANCED

- A. Bellman-Ford Algorithm**
- B. Dijkstra's Algorithm
- C. Prim's Algorithm
- D. Kruskal's Algorithm

Correct Answer: A. Bellman-Ford Algorithm

Explanation: Bellman-Ford relaxes all edges $V-1$ times and can correctly handle negative weight edges, also detecting negative-weight cycles, unlike Dijkstra.

Q24 Which algorithms are used to find a Minimum Spanning Tree (MST)?

ADVANCED

- A. Kruskal's and Prim's Algorithms**
- B. Dijkstra's and Bellman-Ford
- C. DFS and BFS only
- D. Floyd-Warshall only

Correct Answer: A. Kruskal's and Prim's Algorithms

Explanation: Kruskal's (edge-based, using union-find) and Prim's (vertex-based, using a priority queue) are the two classic greedy algorithms for finding an MST.

Q25 What does topological sorting apply to?

ADVANCED

- A. A Directed Acyclic Graph (DAG), producing a linear ordering respecting edge directions
- B. Any undirected graph
- C. Only balanced binary trees
- D. Only weighted graphs

Correct Answer: A. A Directed Acyclic Graph (DAG), producing a linear ordering respecting edge directions

Explanation: Topological sort orders vertices of a DAG such that for every directed edge $u \rightarrow v$, u comes before v — used in task scheduling, build systems, etc. It's undefined for graphs with cycles.

Q26 The Floyd-Warshall algorithm is used to find:

ADVANCED

- A. Shortest paths between ALL pairs of vertices
- B. Shortest path from a single source only
- C. A minimum spanning tree
- D. Whether a graph is bipartite

Correct Answer: A. Shortest paths between ALL pairs of vertices

Explanation: Floyd-Warshall is a dynamic-programming algorithm computing shortest paths between every pair of vertices in $O(V^3)$, suitable for dense graphs.

5. Sorting, Searching & Dynamic Programming Sorting algorithms, DP patterns,

greedy

Q27 Which sorting algorithm is 'stable' by nature (preserves relative order of equal elements)?

INTERMEDIATE

- A. Merge Sort
- B. Quick Sort
- C. Heap Sort
- D. Selection Sort

Correct Answer: A. Merge Sort

Explanation: Merge sort's merging process preserves the original relative order of equal elements, making it stable; standard quicksort, heapsort, and selection sort are not stable.

Q28 What is the space complexity of merge sort?

INTERMEDIATE

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A. $O(n)$

Explanation: Merge sort requires additional $O(n)$ auxiliary space for the temporary arrays used during the merge step (unlike in-place quicksort).

Q29 Which searching algorithm requires the input data to be sorted beforehand?

BASIC

- A. Binary Search
- B. Linear Search
- C. Depth-First Search
- D. Breadth-First Search

Correct Answer: A. Binary Search

Explanation: Binary search relies on repeatedly halving a sorted range; it does not work correctly on unsorted data.

Q30 Dynamic programming is most applicable to problems that exhibit:

INTERMEDIATE

- A. Optimal substructure and overlapping sub-problems
- B. Only overlapping sub-problems with no optimal substructure
- C. Random, unrelated sub-problems
- D. Only problems with a single valid solution path

Correct Answer: A. Optimal substructure and overlapping sub-problems

Explanation: DP requires that an optimal solution can be built from optimal solutions of sub-problems (optimal substructure), and those sub-problems recur (overlapping sub-problems) so results can be cached.

Q31 What is the time complexity of the classic 0/1 Knapsack DP solution with n items and capacity W ?

ADVANCED

- A. $O(nW)$
- B. $O(2^n)$
- C. $O(n \log n)$
- D. $O(W \log n)$

Correct Answer: A. $O(nW)$

Explanation: The standard DP table has $n \times W$ states, each computed in $O(1)$, giving pseudo-polynomial time complexity $O(nW)$.

Q32 A greedy algorithm makes decisions based on:

BASIC

- A. The locally optimal choice at each step, hoping it leads to a global optimum**
- B. Exhaustively trying every possible combination
- C. Random selection
- D. Always backtracking after each choice

Correct Answer: A. The locally optimal choice at each step, hoping it leads to a global optimum

Explanation: Greedy algorithms commit to the best immediate choice without reconsidering, which works for problems with the 'greedy choice property' (e.g., activity selection, Huffman coding) but not universally.

Q33 Which classic problem CANNOT be solved correctly using a simple greedy approach (requires DP instead)?

ADVANCED

- A. 0/1 Knapsack Problem**
- B. Activity Selection Problem
- C. Minimum Spanning Tree (Kruskal's)
- D. Huffman Coding

Correct Answer: A. 0/1 Knapsack Problem

Explanation: 0/1 Knapsack requires considering combinations of items since you can't take fractional items, so a greedy 'best ratio first' approach can give a suboptimal answer — DP is needed for correctness.

Q34 Memoization in dynamic programming refers to:

BASIC

- A. Storing (caching) results of expensive function calls and returning the cached result when the same inputs occur again**
- B. Sorting an array before processing
- C. Removing duplicate elements
- D. Converting recursion into a loop always

Correct Answer: A. Storing (caching) results of expensive function calls and returning the cached result when the same inputs occur again

Explanation: Memoization is the top-down DP technique that caches sub-problem results (often via a hash map or array) to avoid redundant recomputation.